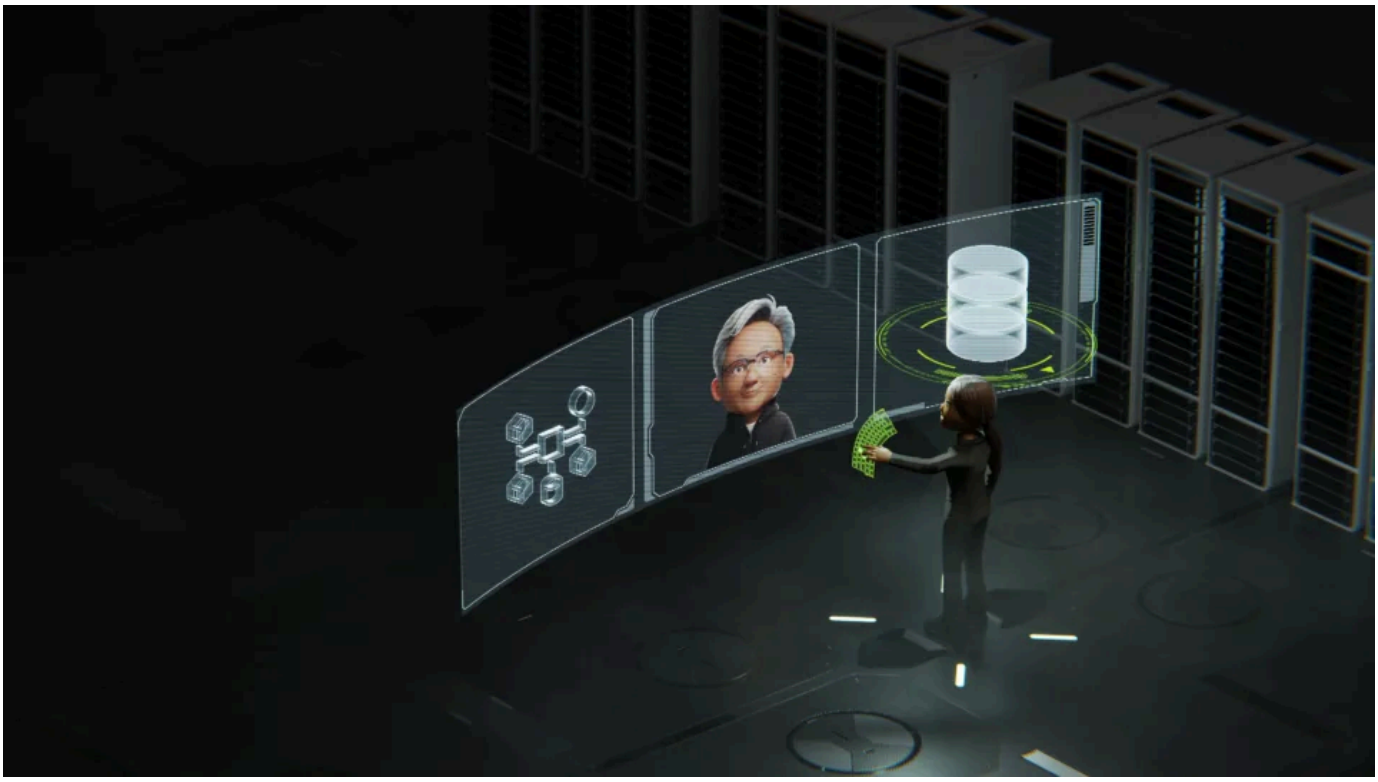
# How to Reduce KV Cache Bottlenecks with NVIDIA Dynamo



Sep 18, 2025

+25 Like Discuss (0)

By Amr Elmeleegy and Akshatha Kamath

AI-Generated Summary

OSS and DeepSeek-R1 rely heavily on attention data—the Key-Value (KV) Cache—to understand and contextualize input prompts, but managing this data efficiently is becoming increasingly difficult.

This post explores how offloading the KV Cache to cost-efficient storage during inference can help reduce inference costs and enhance the user experience. It also explains how recent optimizations in NVIDIA Dynamo make this possible.

# What is the KV Cache?

The KV Cache is a data structure at the core of an LLM's attention mechanism created during the initial phase of inference known as prefill. The KV Cache stores intermediate attention data that helps the model focus on the most relevant parts of the input during the generation or response phase.

However, the KV Cache grows linearly with prompt length and must reside in GPU memory during the generation process for fast access. As models expand context windows, sometimes reaching millions of tokens, KV Cache becomes a serious bottleneck.

# Why is KV Cache a bottleneck for LLM inference?

GPU memory is limited and costly. As the prompt length increases, the KV Cache grows larger, requiring more memory during generation. In use cases like multi-turn conversations, deep research, and code generation, the KV Cache must be retained in memory for extended periods of time. When GPU memory limits are reached, inference systems face trade-offs. They can:

- Evict parts of the KV Cache, which leads to costly recomputation
- Cap the prompt length or context window, reducing model performance
- Add more GPUs, increasing operational costs

Holding large KV Caches in GPU memory for long durations is not scalable and forces providers to choose between cost, latency, and capability.

bottlenecks?

The latest Dynamo release uses KV Cache offloading to enable the instant transfer of KV Cache from limited GPU memory to larger cost-efficient storage. It directly offloads KV Cache from GPU memory to more scalable and affordable storage systems like CPU RAM, local SSDs, or remote network storage. Using NVIDIA NIXL, a low-latency transfer library, Dynamo can quickly move KV Cache blocks between GPU memory and external storage without interrupting inference.

*Figure 1. KV Cache offloading enables the instant transfer of KV Cache from limited GPU memory to larger cost-efficient storage*

# What are the benefits of KV Cache offloading?

With KV Cache offloading, inference service providers can support models with longer context windows without limiting prompt size. Offloading reduces GPU memory usage, allowing clusters

users as discounts for prompts that include cached input tokens.

KV Cache offloading also avoids expensive KV Cache recomputation, resulting in faster response times and a better user experience. In the end, providers benefit from higher throughput and lower cost per token, making their inference services more scalable and efficient.

# When to offload KV Cache for reuse

Offloading KV Cache to CPU or storage is most effective when KV Cache exceeds GPU memory and cache reuse outweighs the overhead of transferring data. It is especially valuable in long-context, high-concurrency, or resource-constrained inference environments such as:

- **Long sessions and multi-turn conversations**: Offloading preserves large prompt prefixes, avoids recomputation, and improves first-token latency and throughput.
- **High concurrency**: Idle or partial conversations can be moved out of GPU memory, allowing active requests to proceed without hitting memory limits.
- **Shared or repeated content**: Reuse across users or sessions (for example, system prompts and templates) increases cache hits, especially with remote or cross-instance sharing.
- **Memory- or cost-constrained deployments**: Offloading to RAM or SSD reduces GPU demand, allowing longer prompts or more users without adding hardware.
- **I/O-optimized platforms**: Environments with high host–device bandwidth (for example, NVLINK C2C) or GPU Direct Storage benefit more, as transfer latency is lower and can overlap with compute.

# How does KV Cache offloading in Dynamo work?

The Dynamo KV Block Manager (KVBM) is the system that powers cache offloading and memory coordination. It is composed of three main layers:

- **Model integration layer:** Connects popular AI inference engines like NVIDIA TensorRT-LLM and vLLM, with support for SGLang coming soon, to the KVBM system. This removes the need for model-specific integrations and enables consistent functionality across different engines.
- **Memory management layer:** Handles how memory is allocated, organized, and reused. It tracks where data lives and enables developers to customize KV Cache offload strategies without impacting the whole system.

across machines and simplifies integration of third-party storage providers through a plugin based system.

*Figure 2. Dynamo KV Block manager interfaces with different components of the LLM inference ecosystem*

By separating memory management from specific model engines and standardizing access to storage, KVBM simplifies integration and scalability. Storage providers no longer need to customize their systems for different inference engines, as KVBM handles the translation. This architecture improves performance, simplifies development, and enables storage and compute to evolve independently.

# How does Dynamo integrate with LMCache?

an open-source system for caching and reusing memory across CPUs, local and remote storage.

LMCache provides a KV caching layer for inference engines such as vLLM. It provides the ability to offload frequently used data like conversation history or prompts from GPU to cost effective storage, and smart eviction and retrieval strategies for high-volume or repetitive workloads. For teams using vLLM, LMCache offers a powerful KV Cache management solution that aligns with the Dynamo open architecture.

# How are storage providers taking advantage of KV Cache Offload?

Vast tested a high-performance integration between NVIDIA Dynamo and the Vast AI OS to enable persistent KV Cache movement between GPU and storage. Using the GPU Direct Storage (GDS) plugin in Dynamo, Vast achieved 35 GB/s throughput to a single NVIDIA H100 GPU, demonstrating full GPU saturation and confirming that storage was not a performance bottleneck.

In a separate test, Vast validated the impact of persistent KV Cache reuse using vLLM and LMCache on an NVIDIA DGX H100 system. Running the Qwen3-32B model with a 130K-token prompt, the system loaded precomputed KV cache from Vast storage rather than recomputing it, reducing Time to First Token (TTFT).

WEKA conducted lab testing to evaluate high-performance KV Cache movement between GPU and storage using NVIDIA Dynamo and a custom NIXL plugin developed and open-sourced by WEKA. The tests demonstrated that the WEKA's Augmented Memory Grid can stream KV Cache from its token warehouse to GPUs at near-memory speeds, reducing TTFT and improving overall token throughput for inference workloads.

Testing was performed using a DGX system with eight H100 GPUs. The setup achieved read throughput up to 270 GB/s across eight GPUs, validating that WEKA's RDMA-based, zero-copy data path can meet the demands of disaggregated inference without becoming a bottleneck.

These test results highlight the potential of KV Cache offload to storage in supporting large-context, high-throughput generative AI workloads in distributed environments.

# the KV Cache

To use KVBM to manage the KV Cache and do KV offloading in vLLM, use the following steps:

```
# start up etcd for KVBM leader/worker registration and discovery
docker compose -f deploy/docker-compose.yml up -d

# build a container containing vllm and kvbm
./container/build.sh --framework vllm --enable-kvbm

# launch the container
./container/run.sh --framework vllm -it --mount-workspace --use-nixl-gds

# enable kv offloading to CPU memory
# 4 means 4GB of CPU memory would be used
export DYN_KVBM_CPU_CACHE_GB=4

# enable kv offloading to disk
# 8 means 8GB of disk would be used
export DYN_KVBM_DISK_CACHE_GB=8

# serve an example LLM model
vllm serve --kv-transfer-config
'{"kv_connector":"DynamoConnector","kv_role":"kv_both",
"kv_connector_module_path": "dynamo.llm.vllm_integration.connector"}'
deepseek-ai/DeepSeek-R1-Distill-Llama-8B

# make a call to LLM
curl localhost:8000/v1/chat/completions   -H "Content-Type: application/json"   -d '{
    "model": "deepseek-ai/DeepSeek-R1-Distill-Llama-8B",
    "messages": [
    {
        "role": "user",
        "content": "In the heart of Eldoria, an ancient land of boundless magic and mysteri
lies the long-forgotten city of Aeloria. Once a beacon of knowledge and power, Aeloria was
beneath the shifting sands of time, lost to the world for centuries. You are an intrepid ex
for your unparalleled curiosity and courage, who has stumbled upon an ancient map hinting t
holds a secret so profound that it has the potential to reshape the very fabric of reality.
will take you through treacherous deserts, enchanted forests, and across perilous mountain
Your Task: Character Background: Develop a detailed background for your character. Describe
motivations for seeking out Aeloria, their skills and weaknesses, and any personal connecti
ancient city or its legends. Are they driven by a quest for knowledge, or a search for lost
    }
    ],
    "stream":false,
    "max_tokens": 30
  }'
```

NVIDIA DEVELOPER Join

To enable metrics collection and view through the Grafana dashboard, use the following steps:

```
# Start the basic services (etcd & natsd), along with Prometheus and Grafana
docker compose -f deploy/docker-compose.yml --profile metrics up -d

# start vllm with DYN_SYSTEM_ENABLED set to true and DYN_SYSTEM_PORT port to 6880.
# NOTE: Make sure port 6880 (for KVBM worker metrics) and port 6881
(for KVBM leader metrics) are available.
DYN_SYSTEM_ENABLED=true DYN_SYSTEM_PORT=6880 vllm serve --kv-transfer-config
'{"kv_connector":"DynamoConnector","kv_role":"kv_both",
"kv_connector_module_path":
"dynamo.llm.vllm_integration.connector"}'
deepseek-ai/DeepSeek-R1-Distill-Llama-8B

# optional if firewall blocks KVBM metrics ports to send prometheus metrics
sudo ufw allow 6880/tcp
sudo ufw allow 6881/tcp
```

View Grafana metrics through `http://localhost:3001` (default login: dynamo/dynamo) and look for the KVBM Dashboard.

# Benchmark KVBM

When vLLM serve is ready, follow these steps to use LMBenchmark to benchmark KVBM performance:

```
git clone https://github.com/LMCache/LMBenchmark.git

# show case of running the synthetic multi-turn chat dataset.
# we are passing model, endpoint, output file prefix and qps to the sh script.
cd LMBenchmark/synthetic-multi-round-qa
./long_input_short_output_run.sh \
    "deepseek-ai/DeepSeek-R1-Distill-Llama-8B" \
    "http://localhost:8000" \
    "benchmark_kvbm" \
    1

# Average TTFT and other perf numbers would be in the output from above cmd
```

To learn more about how to use LMBenchmark, visit the LMCache/LMBenchmark GitHub repo.

Note that if metrics are enabled as mentioned in the previous section, you can observe KV offloading, and KV onboarding in the Grafana dashboard.

# How to get started with Dynamo using LMCache and vLLM

LMCache is enabled by setting the `ENABLE_LMCACHE` environment variable:

- `export ENABLE_LMCACHE=1`

Additional LMCache configuration can be customized through environment variables:

- `LMCACHE_CHUNK_SIZE=256` – Token chunk size for cache granularity (default: 256)
- `LMCACHE_LOCAL_CPU=True` – Enable CPU memory backend for offloading
- `LMCACHE_MAX_LOCAL_CPU_SIZE=20` – CPU memory limit in GB (user can adjust based on available RAM to a fixed value)

For advanced configurations, LMCache supports multiple storage backends:

- **CPU RAM**: Fast local memory offloading
- **Local Storage**: Disk-based persistence
- **Redis**: Distributed cache sharing
- **GDS Backend**: GPU Direct Storage for high throughput
- **InfiniStore/Mooncake**: Cloud-native storage solutions

To get started with Dynamo using LMCache and vLLM, use the following steps:

```
# start up etcd for KVBM leader/worker registration and discovery
docker compose -f deploy/docker-compose.yml up -d

# build a container containing vllm and kvbm
./container/build.sh --framework vllm

# launch the container
./container/run.sh --framework vllm -it --mount-workspace

# run vllm with lmcache in aggregated inference
./components/backends/vllm/launch/agg_lmcache.sh

# run vllm with lmcache in disaggregated inference
./components/backends/vllm/launch/disagg_lmcache.sh
```

# Summary

As LLMs continue to scale, managing the KV Cache during inference has become a major challenge due to limited and costly GPU memory. NVIDIA Dynamo addresses this by enabling KV Cache offloading to more scalable storage options such as CPU RAM, SSDs, and networked storage, powered by the low-latency NIXL transfer library.

Dynamo integrates seamlessly with popular inference engines like vLLM and open source tools like LMCache, enabling efficient cache reuse, reduced recomputation, and better support for long-context and high-concurrency workloads. Storage providers such as Vast and WEKA have successfully integrated with Dynamo, demonstrating how high-throughput storage systems can offload and stream KV Cache effectively without becoming a bottleneck.

These capabilities make KV Cache offloading a practical and scalable solution for reducing inference costs, improving responsiveness, and enabling broader deployment of large-scale generative AI applications. Learn more and get started with Dynamo.

Discuss (0) +25 Like

## Tags

Agentic AI / Generative AI | Data Center / Cloud | Networking / Communications | General | Dynamo | Intermediate Technical | Best Practice | Tutorial | AI Inference | Featured | Inference Performance | LLMs

## About the Authors

**About Amr Elmeleegy**

Amr Elmeleegy is a principal product marketing manager for accelerated computing in the data center, focused on the NVIDIA AI inference platform. Previously, he held business development and product marketing roles at AWS and SAP. He holds an MBA from the UC Berkeley

**About Akshatha Kamath**

Akshatha Kamath is a product manager at NVIDIA enabling efficient memory management for AI inference. She holds an MBA and a master's degree in Data Science for Education from Stanford University. She brings experience developing and scaling ML/AI applications across industries, including AI-powered legal research at LawPavilion in Nigeria.

**View all posts by Akshatha Kamath**

## Comments

Start the discussion at forums.developer.nvidia.com

## Related posts

**Optimizing Inference for Long Context and Large Batch Sizes with NVFP4 KV Cache**

## Introducing New KV Cache Reuse Optimizations in NVIDIA TensorRT-LLM

NVIDIA DEVELOPER Join

## 5x Faster Time to First Token with NVIDIA TensorRT-LLM KV Cache Early Reuse

NVIDIA DEVELOPER

Join

**NVIDIA GH200 Superchip Accelerates Inference by 2x in Multiturn Interactions with Llama Models**

The Next Generation of AI Begins

It all starts here. June 1–4 | Taipei

Register Now

NVIDIA GTC

Sign up for NVIDIA News

Subscribe

Follow NVIDIA Developer

NVIDIA DEVELOPER

Privacy Policy | Your Privacy Choices | Terms of Use | Accessibility | Corporate Policies | Contact

Copyright © 2026 NVIDIA Corporation